

DSCI 511 Quiz 1 Cheat Sheet — 内容稿

函数：可变默认值陷阱

默认参数表达式在函数定义时只创建一次。因此不要把会被修改的对象写作默认值：

```
def add(x, values=[]): # 不要这样写：各次省略 values 的调用共享同一 list
    values.append(x)
    return values
```

用 None 表示调用者没有提供值，并在函数体内创建新的 list：

```
def add(x, values=None):
    if values is None:
        values = []
    values.append(x)
    return values
```

函数：全局名字、mutation 与副作用

- 在函数体中赋值的名字默认是局部名字，函数结束后外部不能使用它。
- 函数可读取外层名字；若要在函数内重新绑定全局名字，必须写 global name，通常应避免。
- 改变传入的可变对象，和重新绑定 parameter，是两回事：

```
def mutate(xs):
    xs.append(0) # 改同一个 list；caller 看得到变化
```

```
def rebind(xs):
    xs = xs + [0] # xs 在此处改绑到新 list；caller 原 list 不变
```

- print、写文件、修改传入的 list/dict、修改全局状态都是副作用。若副作用是函数契约的一部分，docstring 应说明。
- 需要保留输入时，先构造新对象或复制（例如 xs.copy()）；b = a 只是给同一个可变对象另取一个名字，不是复制。

函数：文档与 type hints

Docstring 是紧跟在 def 后的三引号字符串。短函数可一行；较大函数至少说明：行为、参数、返回值、特殊情况/副作用和用法。课程推荐 NumPy/SciPy 风格的 Parameters、Returns、Examples 小节。

```
def repeat(text: str, n: int = 2) -> str:
    """Return text repeated n times."""
    return text * n
```

- text: str, n: int 是 parameter 的 type hints；-> str 是返回值 hint。
- Hints 帮助读者和 IDE；普通 Python 不会因为 hint 自动拒绝其他类型。
- 副作用是接口的一部分时，在 docstring 中明确写出会改变什么。

面向对象（OOP）

核心模型：class、instance、attribute、method

- class 是创建某类对象的蓝图；instance/object 是该 class 创建出的具体对象。isinstance(obj, Class) 检查对象是否属于该 class。
- attribute 是对象状态，如 dog.name；method 是 class 内定义的 function，如 dog.teach("roll")。method 有 () 才调用；没有 () 得到的是 method object。
- __init__ 是创建 instance 时自动执行的 special method。普通 instance method 的第一个 parameter 是 self；调用 obj.method(x) 时，Python 自动把 obj 传给 self。

```
class Dog:
    species = "dog" # class attribute：共享默认值

    def __init__(self, name):
        self.name = name # instance attribute：每个对象自己的值
        self.tricks = [] # 每个对象自己的可变状态

    def teach(self, trick):
        self.tricks.append(trick)

    def description(self):
        return f"{self.name}:"
{self.tricks}"
```

Class attribute 与 instance attribute

- class attribute 写在 __init__ 外，所有 instance 默认共享；改 Dog.species 会影响没有同名 instance attribute 的对象。

- dog.species = "..." 会在该对象上创建或覆盖同名 instance attribute；这通常不是“修改整个 class 的规则”。
- 每个 instance 的可变状态在 __init__ 中创建，如 self.tricks = []。不要把会修改的 list 放成 class attribute，否则所有 instance 共用它。

三类 method

种类	写法/首参	何时用
instance method	无 decorator；self	需要读/改某个 instance 的状态
class method	@classmethod；cls	需要 class 本身；常作 alternative constructor
static method	@staticmethod；无自动 self/cls	与 class 相关、但不需 instance 或 class 状态的 utility

```
class Member:
    def __init__(self, first, last):
        self.first, self.last = first, last

    @classmethod
    def from_csv(cls, text):
        first, last = text.split(",")
        return cls(first, last)

    @staticmethod
    def is_quiz_week(week):
        return week in [3, 5]
```

Alternative constructor 中用 cls(...)，不写死 Member(...)：这样 subclass 继承后仍会构造正确的 class。

继承与 special methods

```
class Student(Member):
    role = "student"

    def __init__(self, first, last, grade):
        super().__init__(first, last) # 初始化 parent 的部分
        self.grade = grade
```

- class Student(Member)：Student 继承 parent/superclass Member；可新增或重写 attribute/method。查找顺序是先 subclass，后沿继承链找 parent。
- 重写 __init__ 时用 super().__init__(...) 复用 parent 初始化，避免复制已有逻辑。
- Double-underscore special (magic) methods 赋予内置语法含义：__len__ -> len(obj)，__eq__ -> ==，__str__ -> print(obj) 的字符串。只在该 class 有合理的长度、相等性或字符串表示时定义。

NumPy：同质多维数组

import numpy as np
np.ndarray 是矩形（每一维长度一致）、通常同一 dtype 的 n 维数组。它适合数值数据；核心写法是向量化，而非逐元素手写 Python loop。

创建与属性

表达式	返回/含义
np.array(data)	从 list/tuple 等创建 ndarray；混合类型可能 coercion 为共同 dtype
np.arange(start, stop, step)	等间隔 array；stop 不含
np.linspace(start, stop, num)	num 个等距值；默认包含 stop
np.full(shape, fill_value)	指定 shape、填满某值
np.zeros(shape)	全 0 array
np.random.randn(d0, d1, ...)	给定维度、元素在 [0, 1) 的随机 float array
a.dtype	元素类型
a.shape	各轴长度的 tuple，例如 (2, 3)
a.size / a.ndim	总元素数 / 轴数
a = np.array([[1, 2, 3], [4, 5, 6]]) # shape (2, 3); ndim 2; size 6 np.arange(0, 10, 2) # [0, 2, 4, 6, 8] np.linspace(0, 1, 5) # [0., 0.25, 0.5, 0.75, 1.]	

```
np.full((2, 3), 4) # 2 rows x 3 columns
```

向量化、数学函数与 axis

- 算术是 element-wise，产生新 array：a + 5、a * b、a ** 2。Python list 不支持 my_list + 5；对 ndarray 的逐元素运算不要手写 loop。
- np.log(a) 对每个元素求自然对数，shape 不变。
- np.sum(a)、np.mean(a)、np.min(a)、np.max(a)：无 axis 时归约全部元素为一个 scalar；对 2D array：

参数	归约方向	结果 shape (原 shape (2, 3))
无 axis	全部元素	scalar
axis=0	沿 rows；每列一个结果	(3,)
axis=1	沿 columns；每行一个结果	(2,)

先写 shape 再推算。可一致地写 np.sum(a)。

多维索引、布尔 mask 与赋值

```
a[1, 2] # row 1, column 2 的 scalar
a[:, 0] # 所有 rows 的第一列；通常少一维
a[0, 1:3] # row 0 的 columns 1、2
positive = a[a > 0] # a > 0 先给同 shape 的 bool mask；再筛选 True 值
```

- 多维索引按轴用逗号分隔：a[row, column]；表示该轴全取。数字 index 通常移除该轴，slice 保留该轴。
- a > 0 是 bool array，不是单个 True/False。
- mask 可原地替换：a[a < 0] = np.nan。a 必须能存 float；int array 先 a = a.astype(float)。
- 这是 mutation：b = a 时 b 是 alias，改 b 也改 a；独立副本用 a.copy()。
- np.nan 是特殊 float；不要用 == np.nan 判断，因为 NaN 不等于自身。

Broadcasting

NumPy 在算术前对齐维度：scalar 广播到每个元素；size 为 1 的维度可扩展。

```
np.array([1, 2, 3]) + 10 # (3,) + scalar -> (3,)
np.ones((2, 3)) + np.array([1, 2, 3]) # (2, 3) + (3,) -> (2, 3)
```

从最右侧比较 dimensions；两维 compatible 当且仅当长度相等，或其中一个为 1。缺少的前导维度视为 1；否则 ValueError (如 (3,) 与 (6,))。先检查 .shape，能运行不代表符合分析意图。

只改 shape，不改数据值

操作	效果与陷阱
np.transpose(a)	交换 axes；2D (r, c) 变 (c, r)
np.reshape(a, new_shape)	同一批元素给新 shape；元素总数必须相同；默认 row-major/C order
a.flatten()	压成 1D 的 copy
np.ravel(a)	压成 1D；尽可能给 view，必要时才 copy

reshape((6, 2)) 不会裁剪或填充，原 array 的 size 必须为 12。若要改扁平结果而不影响原 array，flatten() 的 copy 语义更安全；若避免不必要 copy，要记住 ravel() 的 view 可能共享数据。

快速读题 / 查错清单

- 先标类型和 shape。是 scalar、list、tuple、dict、ndarray 还是 None？二维 array 的 shape 是什么？
- 逐步执行状态变化。标出每次赋值、mutation、循环当前值和 return；不要把 print 当返回值。
- 区分取值、复制和原地修改。x = y、x[:].append()、a[mask] = ... 的效果不同。
- 对索引写出边界。Python 从 0 开始；slice 的 stop 不含；range(stop) 也不含 stop；负 index 从末尾开始。
- 函数对照签名。required/optional 参数是否都绑定？调用返回什么 type？是否有副作用？
- NumPy 先写 shape，再算。尤其是 axis、二维索引、mask、broadcasting。
- 把错误类型当线索。KeyError (dict key)、IndexError (序列位置)、TypeError (操作/类型不匹配)、ValueError (值/shape 不合适)。